VoIP Recording PoC

Goal: Demonstrate VoIP stream capture by hooking Android's `audioserver` (specifically `libaudioflinger.so`) and recording uplink (mic) and downlink (speaker) PCM buffers when the system is in **communication** mode.

Project Design & Methodology

Architecture:

- **Controller App (Android)**
  UI with 9 buttons to manage SELINUX policy setup, payload injection, data movement, and playback.
- **Payload (`libaudiohook.so`)**
  C++ shared library (built as part of the app project) that hooks internal `AudioFlinger` methods using **Dobby** and writes raw PCM buffers to a session directory.
- **Post-Processing**
  Additional tool, `tools/audioparser.py` converts raw `.ac(Downlink)` / `.bc(Uplink)` files to WAV. The app's file manager can also convert & play in-app.
- **Research Utilities**
  Frida scripts for initial function tracing & offset discovery; analysis files for symbol/offset and other snippets.

Controller App

This Android application provides a graphical interface for managing VoIP recording through the `audioserver`. It integrates with a native payload that hooks into the Android audio stack to capture microphone and speaker audio from VoIP applications.

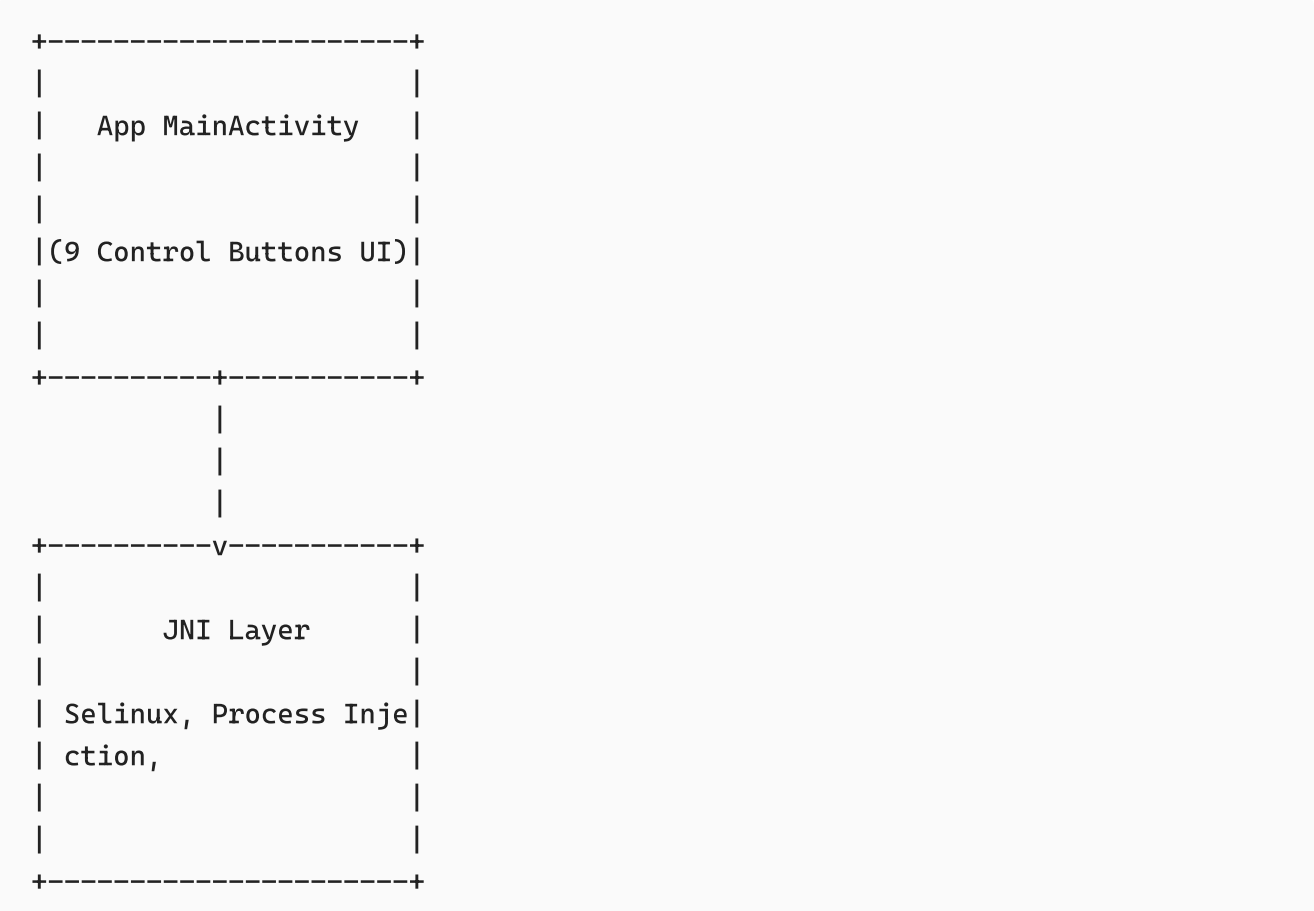
The app serves as a control center, allowing you to inject required SELinux policies, start/stop monitoring, manage recorded data, and play captured VoIP audio files.

Features

The main activity contains **9 control buttons**:

1. **Inject Policies**
   Injects the required SELinux policies for `audioserver` and this app to work together.
2. **Start Monitoring**
   Injects the shared library payload into `audioserver`, enabling VoIP audio capture by hooking `RecordTrack` and `PlaybackTrack`. Uses [AndKittyInjector] for Process Injection.

3. **Stop Monitoring**
   Restarts the `audioserver` process, stopping monitoring and removing injected hooks.
4. **Copy Data**
   Copies captured **raw PCM audio** from temp directory `/data/local/tmp/voip` to `/sdcard/voip`.
   - Cleans up the temporary directory afterward.
5. **View Logs**
   Displays logs related to injection, hooking, and recording status.
6. **Enable SELinux**
   Restores SELinux enforcing mode.
7. **Disable SELinux**
   Sets SELinux to permissive mode, Use when injected policies fail.
8. **Play Audios**
   Opens the custom **VoIP File Manager** (/sdcard/voip/):
   - Converts raw PCM files into **WAV format** in-memory.
   - Plays the converted audio directly in-app.
9. **Delete Data**
   Deletes all audio data from both `/data/local/tmp/voip` and `/sdcard/voip`.
   NOTE: After delete data, we need to reinject into `audioserver` for directory creation to save data.



Hooking Payload

This shared library is injected into `audioserver` to detect `AUDIO_MODE_IN_COMMUNICATION` and capture VoIP audio streams directly from Android's audio stack for predefined set of package names.

It uses the Dobby inline hooking framework to intercept critical methods inside `libaudioflinger.so`.
Both **uplink (microphone)** and **downlink (speaker)** audio streams are captured and written to session files for later processing.

Hooking Points

The library installs hooks on the following functions inside `AudioFlinger` and related classes:

Function	Purpose
`AudioFlinger::setMode`	Tracks transitions into/out of `IN_COMMUNICATION` mode (VoIP active).
`RecordTrack::getNextBuffer`	Captures microphone (uplink) audio frames.
`PlaybackThread::Track::getNextBuffer`	Captures speaker (downlink) audio frames.
`Track::stop`	Closes and finalizes downlink file.
`RecordTrack::stop`	Closes and finalizes uplink file.

Target Applications

The library filters streams based on **UID → Package mapping**.

Currently monitored apps:

- `com.whatsapp`
- `com.whatsapp.w4b` (WhatsApp Business)
- `org.thoughtcrime.securesms` (Signal)
- `org.telegram.messenger`
- `org.telegram.messenger.web`

You can add other applications also iniside `audioserver_hook.cpp`:

```
static const char* kTargetPackages[] = {
    "com.whatsapp",
    "com.whatsapp.w4b",
    "org.thoughtcrime.securesms",
    "org.telegram.messenger",
    "org.telegram.messenger.web"
};
```

File Output

- A **session directory** is created at injection into `audioserver` under: `/data/local/tmp/voip/audioserver_session_<timestamp>/`
- Files are first created with a `.tmp` **extension**, then renamed on track stop:
  - **Uplink (mic)** → `<packageName>_<sampleRate>_<timestamp>.bc`
  - **Downlink (speaker)** → `<packageName>_<sampleRate>_<timestamp>.ac`
- Example session:

```
/data/local/tmp/voip/audioserver_session_1725389200/
├── com.whatsapp_48000_1725389201_123456.ac
├── com.whatsapp_48000_1725389201_123789.bc
```

Offsets

Since target library is stripped and methods are not exported in `libaudioflinger.so`, we need exact offsets of methods and other related components for successful hooking.

We can extract those offsets from static analysis using IDA-PRO or from dynamic analysis using frida.
Once offsets are known we can replace them inside `offsets.h` header file.

Required Offsets:

Methods:

1. AUDIOFLINGER_SETMODE_OFFSET
   - Method: `android::AudioFlinger::setMode(audio_mode_t)`
   - Mangled name: `_ZN7android12AudioFlinger7setModeE11audio_mode_t`
2. RECORDTRACK_GETNEXTBUFFER_OFFSET
   - Method: `android::AudioFlinger::RecordThread::RecordTrack::getNextBuffer(android::AudioBufferProvider::Buffer *)`
   - Mangled name: `_ZN7android12AudioFlinger12RecordThread11RecordTrack13getNextBufferEPNS_19AudioBufferProvider6BufferE`
3. TRACK_GETNEXTBUFFER_OFFSET
   - Method: `android::AudioFlinger::PlaybackThread::Track::getNextBuffer(android::AudioBufferProvider::Buffer *)`
   - Mangled name: `_ZN7android12AudioFlinger14PlaybackThread5Track13getNextBufferEPNS_19AudioBufferProvider6BufferE`

4. TRACK_STOP_OFFSET
    - Method: `android::AudioFlinger::PlaybackThread::Track::stop(void)`
    - Mangled name: `_ZN7android12AudioFlinger14PlaybackThread5Track4stopEv`
5. RECORDTRACK_STOP_OFFSET
    - Method: `android::AudioFlinger::RecordThread::RecordTrack::stop(void)`
    - Mangled name: `_ZN7android12AudioFlinger12RecordThread11RecordTrack4stopEv`

Other Offsets:

1. SAMPLERATE_OFFSET
2. UID_OFFSET
3. ISOUT_OFFSET
4. FORMAT_OFFSET
5. CHANNELMASK_OFFSET
6. ATTR_OFFSET

These can be obtained from static analysis.

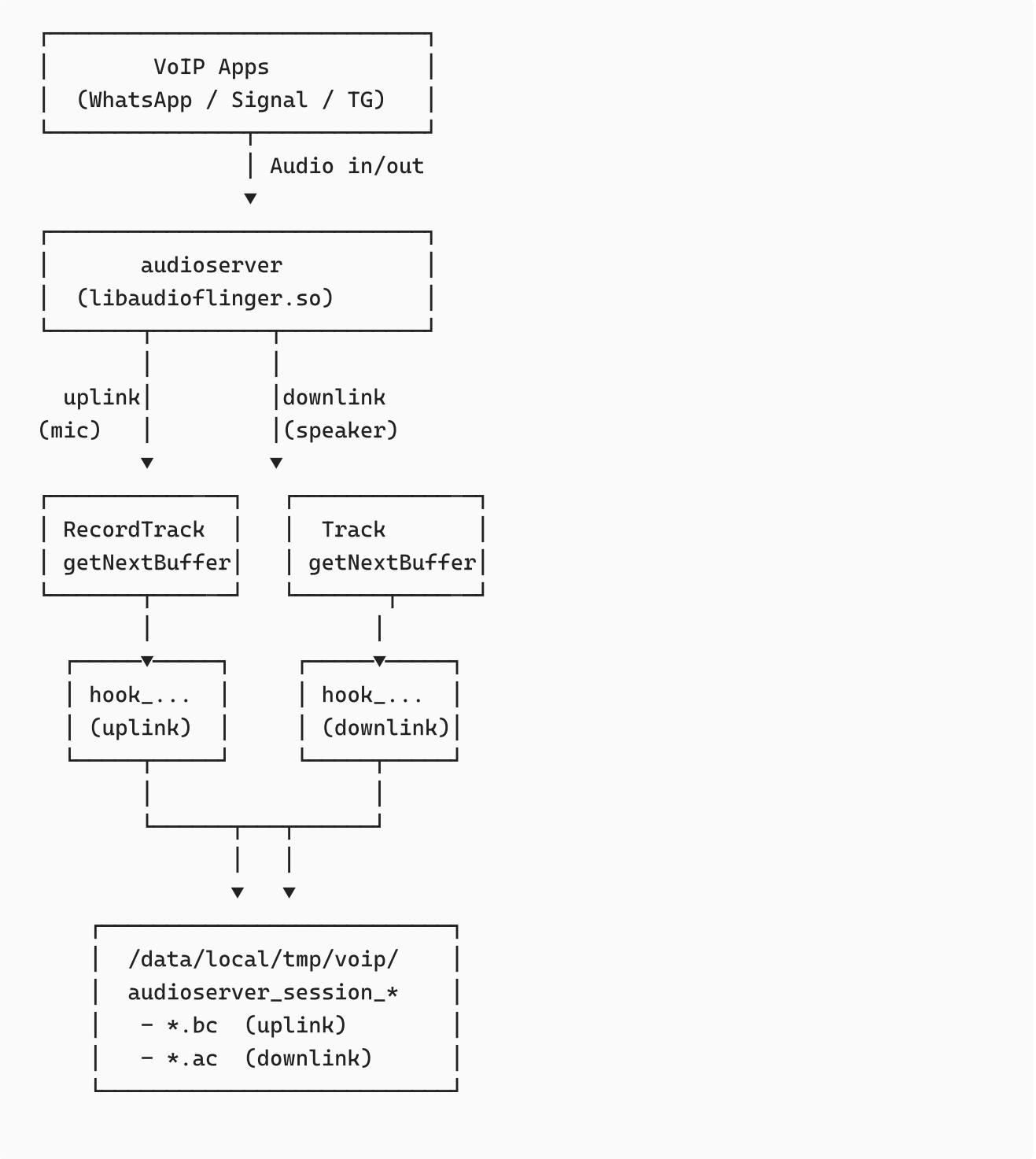
- Library - `libaudioflinger.so`
- Method - `android::AudioFlinger::ThreadBase::TrackBase::TrackBase`

```
TrackBase::TrackBase(
        IAfThreadBase *thread,
            const sp<Client>& client,
            const audio_attributes_t& attr,
            uint32_t sampleRate,
            audio_format_t format,
            audio_channel_mask_t channelMask,
            size_t frameCount,
            void *buffer,
            size_t bufferSize,
            audio_session_t sessionId,
            pid_t creatorPid,
            uid_t clientUid,
            bool isOut,
            const alloc_type alloc,
            track_type type,
            audio_port_handle_t portId,
            std::string metricsId)
```

Notes

- Only **IN_COMMUNICATION** mode is captured. Normal audio (music, ringtone, etc.) is ignored.

- Special handling for **Telegram** and **Signal** enforces correct sample rates (Telegram: 48000 Hz, Signal: 8000 Hz) to ignore other unnecessary audios like phone ring and phone disconnect sounds.
- PCM is stored raw; the controller app later converts PCM → WAV for playback.



Target Function Identification

Process used:

1. Dynamic tracing - Frida:
   - Used Frida to trace all function calls during a VOIP Call
   - Filtered all functions that are responsible for audio buffers handling.

- Then created Frida scripts to capture and handle audio buffers.

2. AOSP review & class mapping
   - `AudioFlinger` manages playback/capture via `PlaybackThread` & `RecordThread`.
   - Per-client audio is represented by `PlaybackThread::Track` (downlink) and `RecordThread::RecordTrack` (uplink).
3. Filtering to App specific streams
   - Used UID→package mapping to restrict to target apps.
4. Stability and offsets
   - Final native payload resolves addresses via base + **hardcoded offsets** (kept in `app/src/main/cpp/include/offsets.h`) to survive stripped symbols.

Detecting `MODE_IN_COMMUNICATION`

We hook `AudioFlinger::setMode(int mode)` and maintain a global state:

- On each call, we log the mode change and update `g_currentAudioMode`.
- Hooks (in both mic and speaker hooks) check `g_currentAudioMode == AUDIO_MODE_IN_COMMUNICATION`.
- This ensures we only dump buffers while a VoIP session is active, avoiding music/ringtones/device playback/mic recordings etc.

Building & Running the PoC

Project Layout

```
.
├── Android_App/
|   ├── app/                            # Android Studio project (App +
Payload)
|   |   ├── src/
|   |   |   ├── main/
|   |   |   |   ├── java/               # Controller App Java/Kotlin sources
|   |   |   |   ├── cpp/                # Payload source (built with the app)
|   |   |   |   |   └── include/
|   |   |   |   |       └── offsets.h  # Function offsets for tested builds
├── tools/
|   ├── audioparser.py              # Converts .ac/.bc raw files → WAV
|   ├── frida/                      # Frida scripts for initial hook
discovery
|   |   ├── host_audioserver.py
|   |   ├── agent_audioserver.js
├── app-release.apk                  # Signed release apk
└── README.md                        # Root README (design, build, run
instructions)
```

Prerequisites

- Android Studio + NDK
- Rooted **emulator** (e.g., rootAVD) or **test device** you own (e.g., Magisk)
- Test VoIP apps installed (same ones you own/administer)
- Python for `audioparser.py`

Build the App (which also builds the payload)

1. Open `app/` in Android Studio.
2. Ensure NDK is installed (SDK Manager → SDK Tools → NDK).
3. Build **Debug** variant. The Gradle/CMake config will compile the native payload (`libaudiohook.so`) as part of the project.

Install on Test Target

`adb install -r app/build/outputs/apk/debug/app-debug.apk`
or Install `app-release.apk` provided:
`adb install -r app-release.apk`

> If you separate artifacts, you can also push the shared library for testing:
> `adb push app/src/main/jniLibs/arm64-v8a/libaudiohook.so /data/local/tmp/`

Run the PoC (Emulator/Owned Device)

1. **Launch the Controller App** -> grant root when prompted.
2. Tap **"Inject Policies"**
   Prepares SELinux rules required for the app and `audioserver` interaction in your test environment.
3. Tap **“Start Monitoring”**
   The app uses **AndKittyInjector** to inject the payload into `audioserver` .
4. Place a **test VoIP call** (e.g., your own WhatsApp/Signal/Telegram account).
5. Tap **“Stop Monitoring”**
   Restarts `audioserver` .
6. Tap **“Copy Data”**
   Moves session files from `/data/local/tmp/voip/…` -> `/sdcard/voip/` (and clears the temp directory).
7. **Playback**
   - In-app: **“Play Audios”** converts PCM -> WAV and plays.
   - Or offline:

     ```
     adb pull /sdcard/voip/
      python3 tools/audioparser.py voip/
     ```

> Additional buttons:

> - **“View Logs”**: See injector/hook status.
> - **“Enable SELinux” / “Disable SELinux”**: Toggle enforcing/permissive in your **lab** if policies are too strict.
> - **“Delete Data”**: Clear both `/data/local/tmp/voip/` and `/sdcard/voip/`.

Tested Versions & Devices

- **Emulator**: AVD **API 34 (x86_64)** - Android 14 - rooted with **rootAVD**
- **Device**: **Samsung Galaxy S22** (ARM64) - Android 14 - Build **S901EXXSCEYB1** - rooted with **Magisk**

Tested Apps

- Whatsapp
- Whatsapp Business
- Signal
- Telegram

Notable Challenges & Solutions

1. **Stripped symbols**
   - **Issue:** Symbols cannot be located directly via `dlsym`.
   - **Fix:** Use Offsets from Static analysis.
2. **Different Symbol/Offset Across Builds**
   - **Issue:** Stripped symbols vary by build; direct `dlsym` may fail.
   - **Fix:** Use base + **hardcoded offsets** for tested builds (`offsets.h`), validated with Frida symbol dumps and IDA-PRO.
3. **Mode Detection Accuracy**
   - **Issue:** Recording outside VoIP sessions yields noisy data.
   - **Fix:** Hooks captures on `AudioFlinger::setMode(IN_COMMUNICATION)` ; log transitions; store a global mode flag.
4. **SELinux**
   - **Issue:** Writing to temp directories & IPC with `audioserver` can be blocked.
   - **Fix:** Provide a **Inject Policies** step to add necessary rules; expose **Enable/Disable SELinux** toggles for debugging in test environments.
5. **File Lifecycle & Renaming**
   - **Issue:** Avoiding partial/corrupt outputs on thread stop.
   - **Fix:** Write to `.tmp` first; on `stop()` rename to `.ac` (downlink) or `.bc` (uplink).
6. No PLT Hooks in `libaudioflinger.so`
   - **Issue:** The target functions (`RecordTrack::getNextBuffer` , `Track::getNextBuffer` , etc.) are **not exported** and `libaudioflinger.so` has no PLT/GOT entries for them.

    - We could not use standard `dlsym()` or PLT-based hooking mechanisms.
- **Fix:** We used Dobby for inline function hooking.
    - Hooks are installed at runtime using the formula:

```
target_addr = base_addr(libaudioflinger) + offset
DobbyHook(target_addr, hook_fn, &orig_fn)
```

References (Open-Source & Docs)

- **Dobby (Inline Hooking Framework)** - https://www.github.com/jmpews/Dobby
- **AndKittyInjector (Android process injector)** - https://github.com/MJx0/AndKittyInjector
- **AOSP**
    - `frameworks/av/services/audioflinger` (AudioFlinger, PlaybackThread, RecordThread)
    - `system/media/audio` (audio formats, attributes, channel masks)